Possible Techniques for Improving DES

- DES is vulnerable nowadays to a brute-force attack
- It is replaced as a standard by AES
- There has been interest to provide another algorithm during the
- transition to AES
- Also interest to preserve the existing investment in software and
- hardware, increasing the security
- **Solution: iterated version of DES** Multiple enciphering with DES

Multiple Encryption and Triple DES

★ Drawbacks of DES.

★ Potential vulnerability of DES to a brute-force attack.

★ AES – Completely a new algorithm.

★ Another alternative.

★ Use of multiple encryption with DES and multiple keys.

Advanced Encryption Standard

- Proposed successor to DES
- DES drawbacks
  - algorithm designed for 1970s hardware implementation
  - performs sluggishly in software implementations
    - 3DES is 3 times slower due to 3 rounds
  - 64 bit blocksize needs to be increased to spped things up
- AES Overview
  - 128, 192, 256 bit blocksize (128 bit likely to be most common)
  - Not a Feistal structure, process entire block in parallel
  - 128 bit key, expanded into 44, 32bit words with 4 words used for each round

Obvious solution: double DES

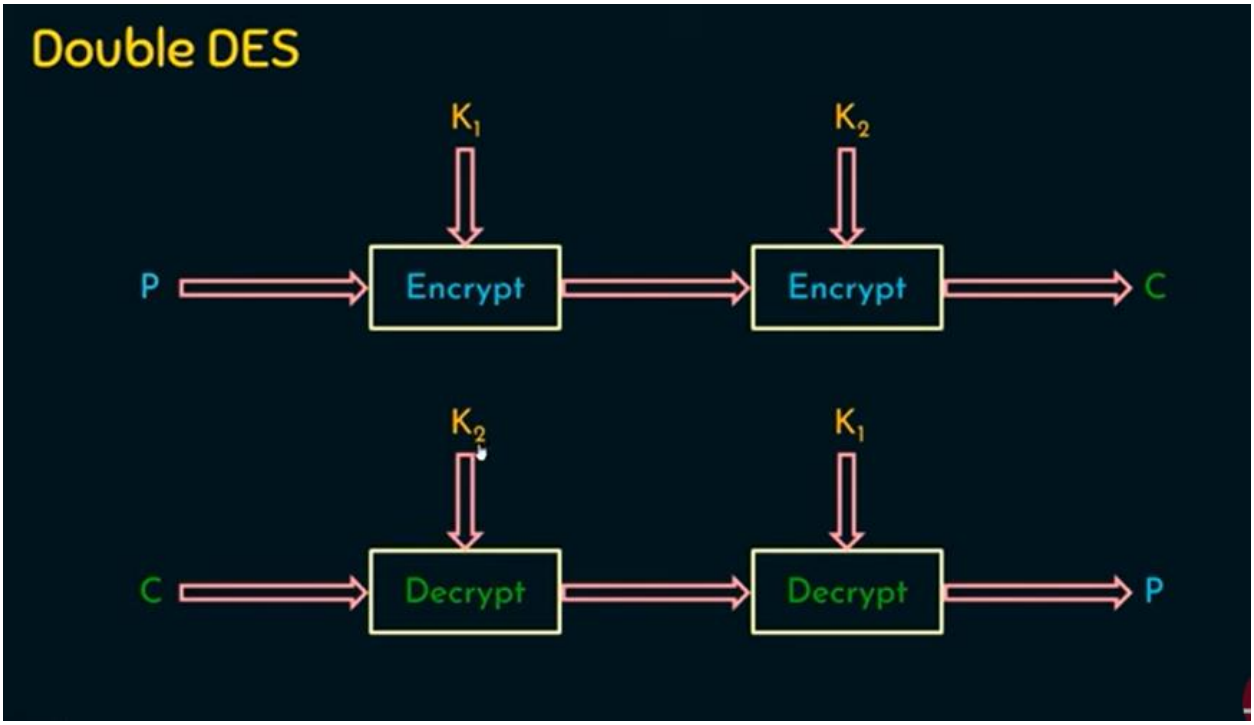
- Use two keys K1, K2 and encrypt the text using the two keys

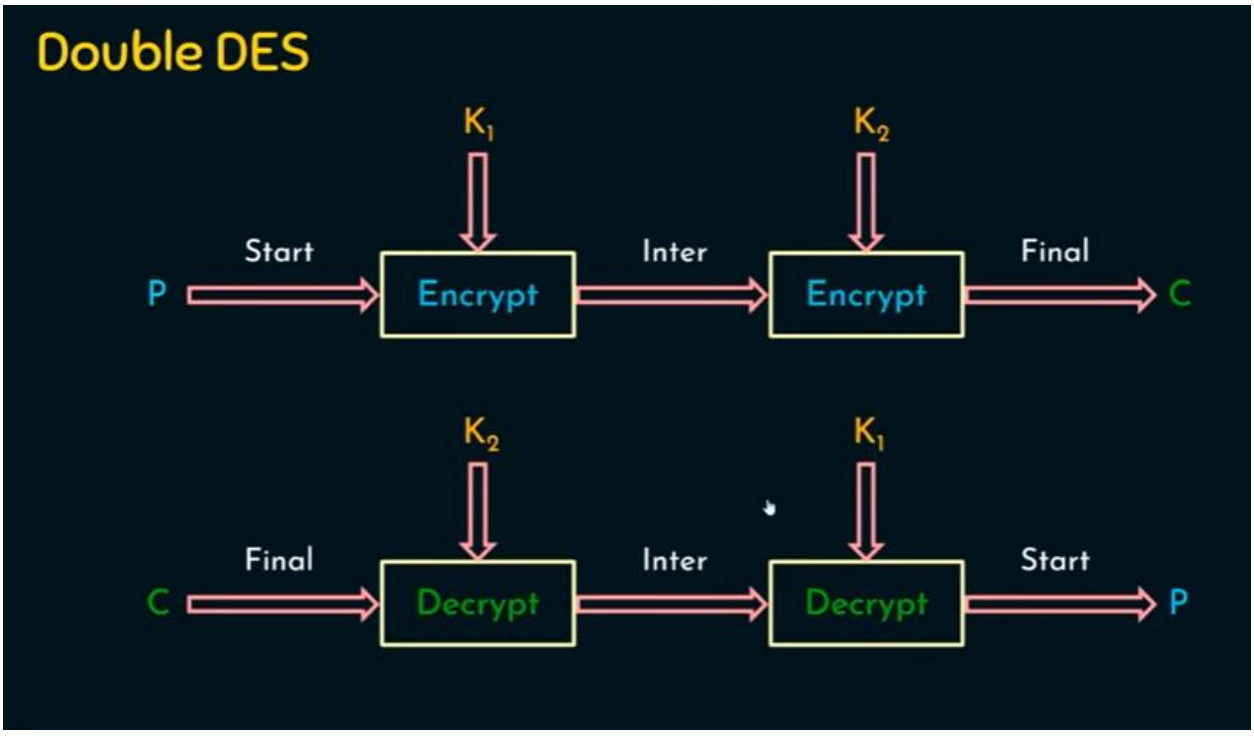
$$C=E_{K2}(E_{K1}(P))$$

- To decrypt simply use DES decryption twice

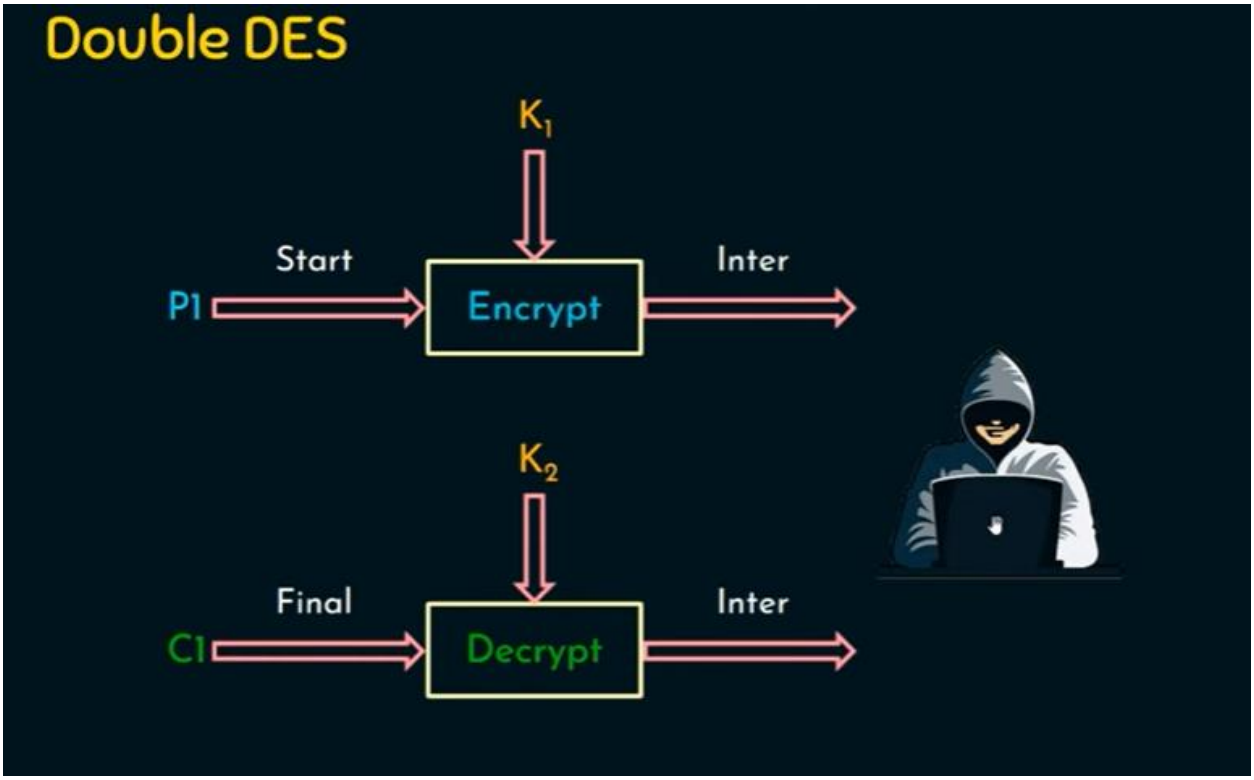
$$P=D_{K1}(D_{K2}(C))$$

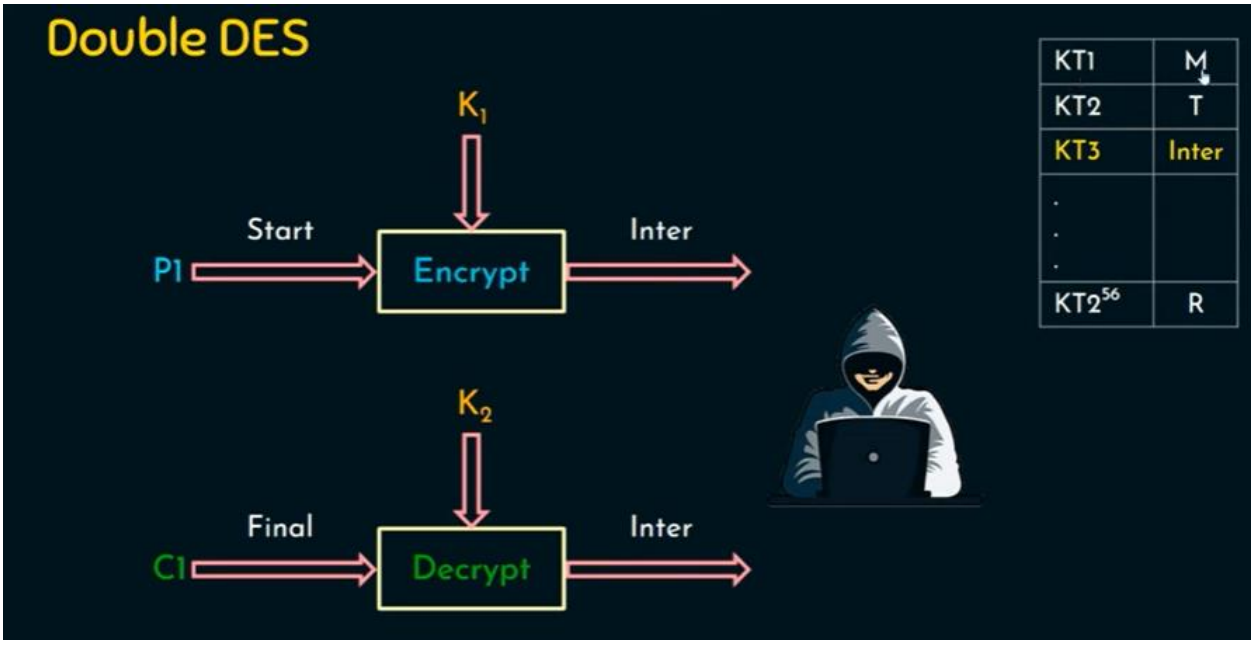
- The scheme involves now a key of 112 bits which should make it much more secure than DES, at least in principle

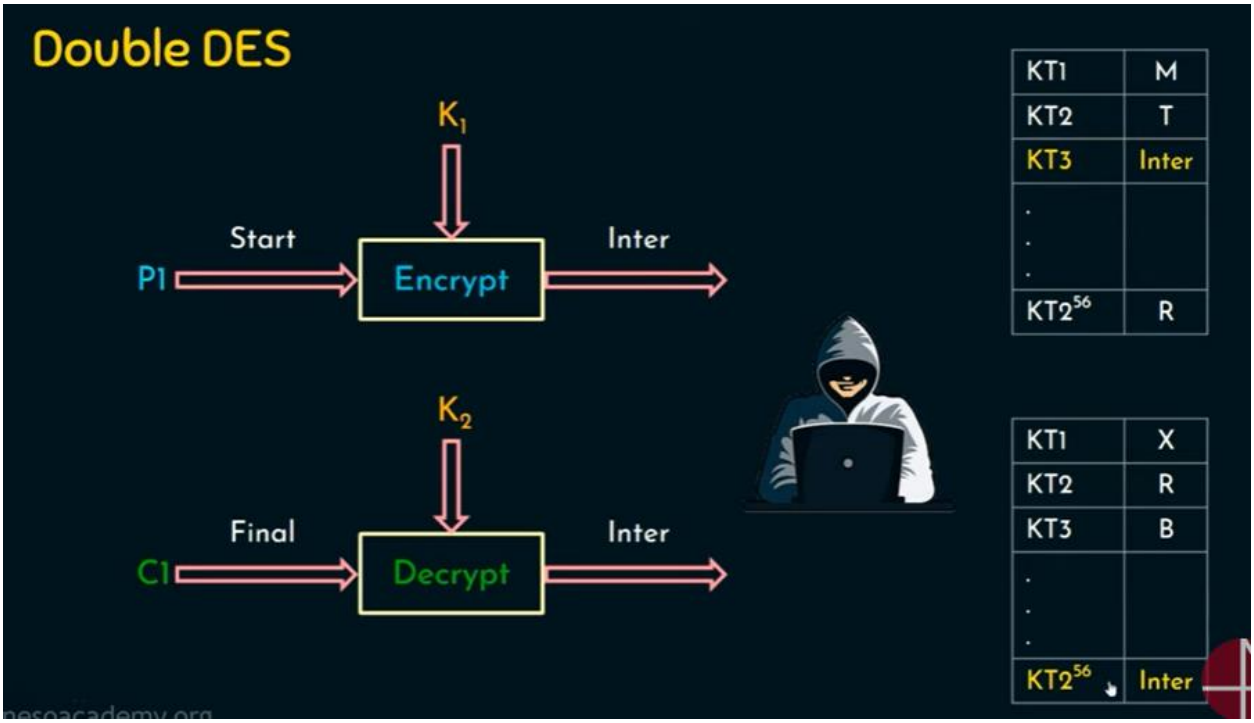

Double DES
K1
K2
P
Encrypt
Encrypt
C
K2
K1
C
Decrypt
Decrypt
P


Double DES
K1
K2
Start
Inter
Final
P
Encrypt
Encrypt
C
K2
K1
Final
Inter
Start
C
Decrypt
Decrypt
P

Meet in the middle attack







Why Triple-DES?

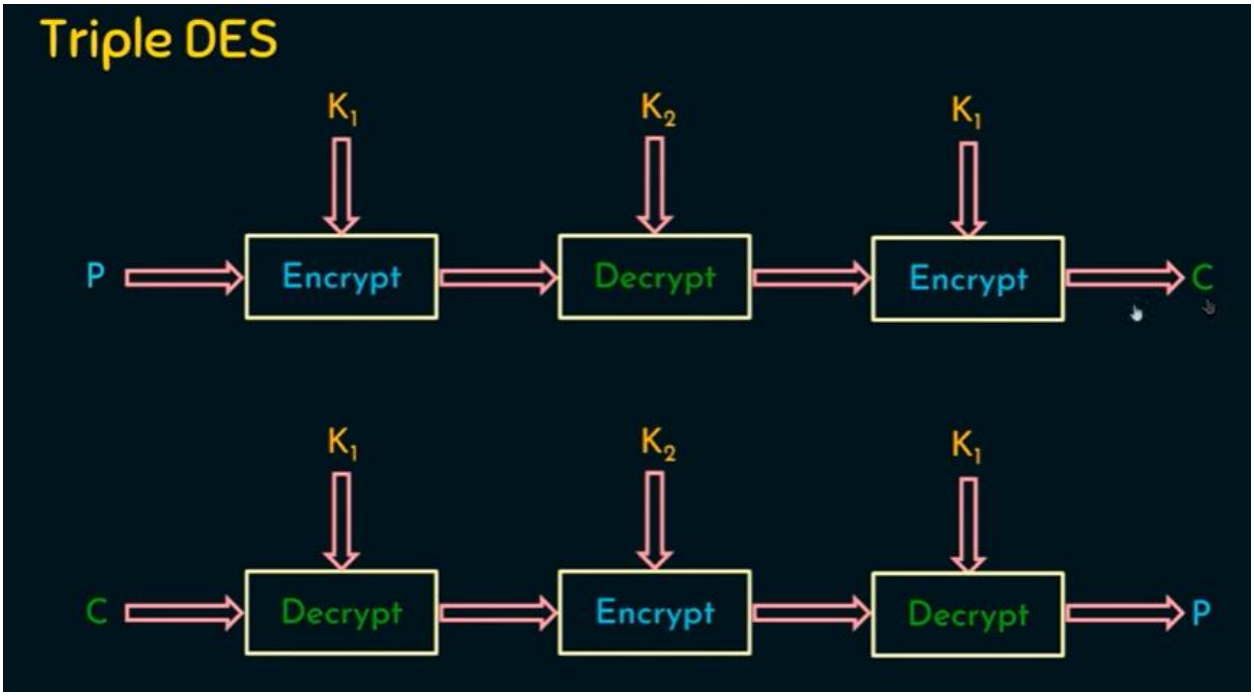
- why not Double-DES?
  - NOT same as some other single-DES use, but have
- meet-in-the-middle attack
  - works whenever use a cipher twice
  - since $X = E_{K1}[P] = D_{K2}[C]$
  - attack by encrypting P with all keys and store
  - then decrypt C with keys and match X value
  - can show takes o (2^{56}) steps

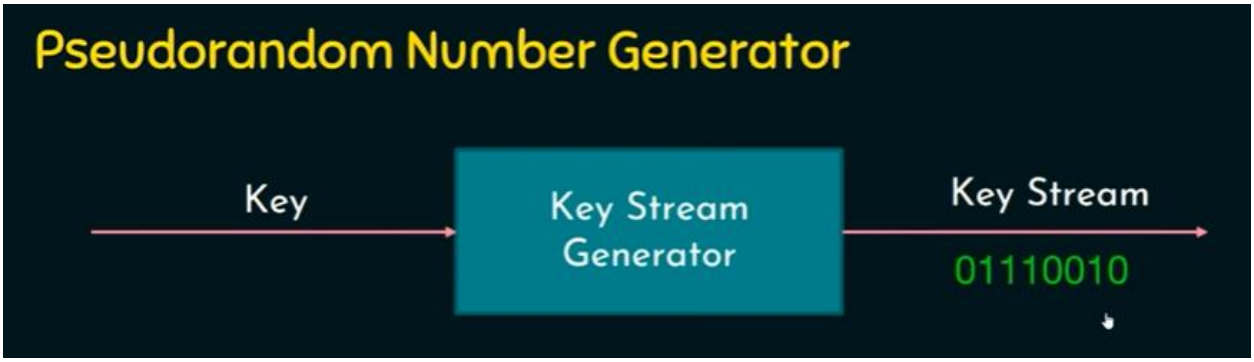
Triple-DES with Two-Keys

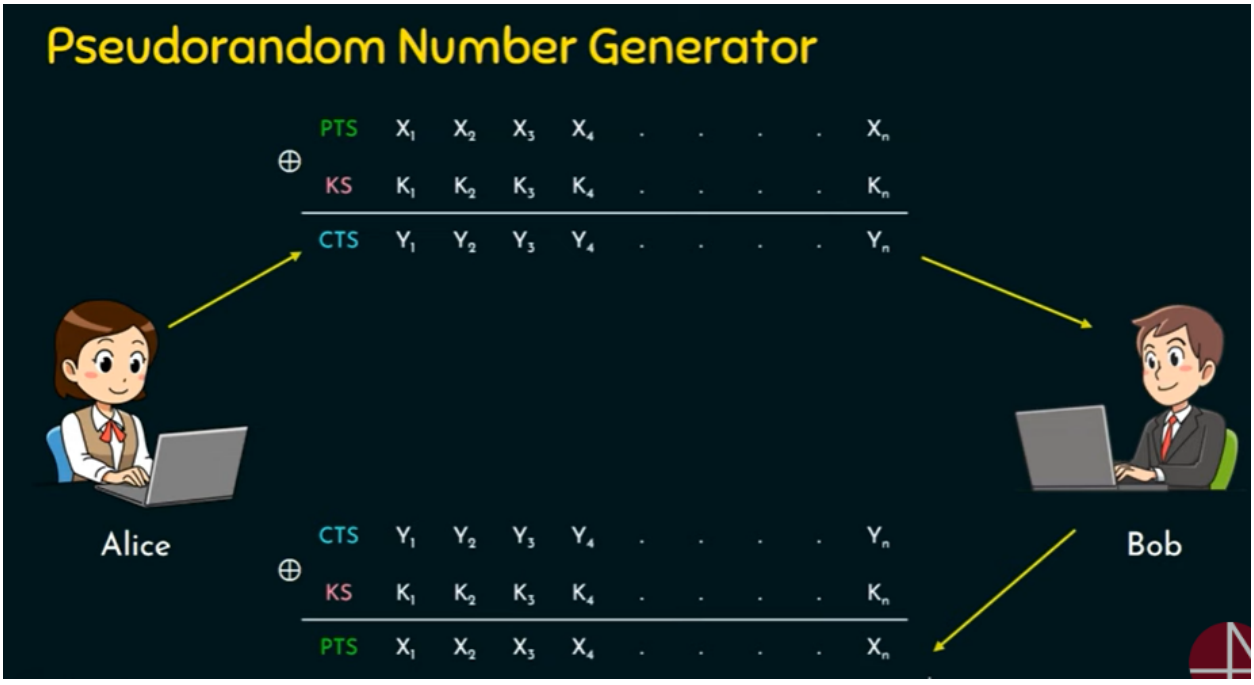
- hence must use 3 encryptions
  - would seem to need 3 distinct keys
  - Key of 56 X 3 = 168 bits seems too long
- but can use 2 keys with E-D-E sequence
  - $C = E_{K1}[D_{K2}[E_{K1}[P]]]$
  - No cryptographic significance to the use of D in the second step
- standardized in ANSI X9.17 & ISO8732
- no current known practical attacks
  - some are now adopting Triple-DES with three keys for greater security

Triple-DES with Three-Keys

- although are no practical attacks on two-key Triple-DES have some indications
- can use Triple-DES with Three-Keys to avoid even these
  - $C = E_{K3}[D_{K2}[E_{K1}[P]]]$
- has been adopted by some Internet applications




Pseudorandom Number Generator
Key
Key Stream Generator
Key Stream
01110010


Pseudorandom Number Generator
PTS X1 X2 X3 X4 . . . . Xn
⊕
KS K1 K2 K3 K4 . . . . Kn
CTS Y1 Y2 Y3 Y4 . . . . Yn
Alice
Bob
CTS Y1 Y2 Y3 Y4 . . . . Yn
⊕
KS K1 K2 K3 K4 . . . . Kn
PTS X1 X2 X3 X4 . . . . Xn

Pseudorandom Number Generator

★ Plaintext : X_i

★ Key Stream : K_i

★ Ciphertext : Y_i

Encryption (Y_i) : $X_i \oplus K_i$

Decryption (X_i) : $Y_i \oplus K_i$

K_i is a truly randon bit.

This stream cipher is referred to as One Time Pad (Perfect Secrecy).

Pseudorandom Number Generator

★ Stream cipher.

★ Key stream generator.

★ Truly random sequence.

★ P(0) = P(1).

★ Shannon notion of perfect secrecy.

★ Generating truly random sequence is impractical.

★ Pseudorandom sequence.

★ A good stream cipher – close to truly random sequence.

★ Randomness.

RSA Algorithm

Key Generation

Select p, q	p and q both prime
Calculate $n = p \times q$	
Calculate $\phi(n) = (p-1)(q-1)$	
Select integer e	$\gcd(\phi(n), e) = 1;\ 1 < e < \phi(n)$
Calculate d	$d \equiv e^{-1} \bmod \phi(n)$
Public key	$KU = \{e, n\}$
Private key	$KR = \{d, n\}$

Encryption

Plaintext	$M < n$
Ciphertext	$C = M^e \pmod{n}$

Decryption

Ciphertext	C
Plaintext	$M = C^d \pmod{n}$

RSA Algorithm

Introduction

Ron Rivest, Adi Shamir and Len Adleman have developed this algorithm (Rivest-Shamir-Adleman). It is a block cipher which converts plain text into cipher text and vice versa at receiver side.

RSA Algorithm Steps

Step-1: Select two prime numbers p and q where $p \neq q$.

Step-2: Calculate $n = p * q$.

Step-3: Calculate $\Phi(n) = (p-1) * (q-1)$.

Step-4: Select e such that, e is relatively prime to $\Phi(n)$, i.e. $(e, \Phi(n)) = 1$ and $1 < e < \Phi(n)$

Step-5: Calculate $d = e^{-1} \bmod \Phi(n)$ or $ed = 1 \bmod \Phi(n)$.

Step-6: Public key = {e, n}, private key = {d, n}.

Step-7: Find out cipher text using the formula,

$C = P^e \bmod n$ where, $P < n$ where C = Cipher text, P = Plain text, e = Encryption key and n=block size.

Step-8: $P = C^d \bmod n$. Plain text P can be obtain using the given formula. where, d = decryption key

RSA algorithm explanation with example step by step

Step – 1: Select two prime numbers p and q where $p \neq q$.

Example, Two prime numbers p = 13, q = 11.

Step – 2: Calculate $n = p * q$.

Example, $n = p * q = 13 * 11 = 143$.

Step – 3: Calculate $\Phi(n) = (p-1) * (q-1)$.

Example, $\Phi(n) = (13 - 1) * (11 - 1) = 12 * 10 = 120$.

Step – 4: Select e such that, e is relatively prime to $\Phi(n)$, i.e. $(e, \Phi(n)) = 1$ and $1 < e < \Phi(n)$.

Example, Select e = 13, gcd (13, 120) = 1.

Step – 5: Calculate $d = e^{-1} \bmod \Phi(n)$ or $e * d = 1 \bmod \Phi(n)$

Example, Finding d: $e * d \bmod \Phi(n) = 1$

$13 * d \bmod 120 = 1$

(How to find: $d * e = 1 \bmod \Phi(n)$

$d = ((\Phi(n) * i) + 1) / e$

$d = (120 + 1) / 13 = 9.30$ (∵ i = 1)

$d = (240 + 1) / 13 = 18.53$ (∵ i = 2)

$d = (360 + 1) / 13 = 27.76$ (∵ i = 3)

$d = (480 + 1) / 13 = 37$ (∵ i = 4))

Step – 6: Public key = {e, n}, private key = {d, n}.

Example, Public key = {13, 143} and private key = {37, 143}.

Step – 7: Find out *cipher text* using the formula, $C = P^e \bmod n$ where, $P < n$.

Example, Plain text P = 13. (Where, $P < n$)

$C = P^e \bmod n = 13^{13} \bmod 143 = 52.$

Step – 8: $P = C^d \bmod n$. Plain text P can be obtain using the given formula.

Example, Cipher text C = 52

$P = C^d \bmod n = 52^{37} \bmod 143 = 13.$

Diffie-Hellman algorithm:

The Diffie-Hellman algorithm is being used to establish a shared secret that can be used for secret communications while exchanging data over a public network using the elliptic curve to generate points and get the secret key using the parameters.

- For the sake of simplicity and practical implementation of the algorithm, we will consider only 4 variables, one prime P and G (a primitive root of P) and two private values a and b.
- P and G are both publicly available numbers. Users (say Alice and Bob) pick private values a and b and they generate a key and exchange it publicly. The opposite person receives the key and that generates a secret key, after which they have the same secret key to encrypt.

Step-by-Step explanation is as follows:

Alice	Bob
Public Keys available = P, G	Public Keys available = P, G
Private Key Selected = a	Private Key Selected = b
Key generated = $x = G^a mod P$	Key generated = $y = G^b mod P$
Exchange of generated keys takes place	
Key received = y	key received = x
Generated Secret Key = $k_a = y^a mod P$	Generated Secret Key = $k_b = x^b mod P$
Algebraically, it can be shown that $k_a = k_b$	
Users now have a symmetric secret key to encrypt	

Example:

```
Step 1: Alice and Bob get public numbers P = 23, G = 9
Step 2: Alice selected a private key a = 4 and
        Bob selected a private key b = 3
Step 3: Alice and Bob compute public values
Alice:    x =(9^4 mod 23) = (6561 mod 23) = 6
        Bob:    y = (9^3 mod 23) = (729 mod 23)  = 16
Step 4: Alice and Bob exchange public numbers
Step 5: Alice receives public key y =16 and
        Bob receives public key x = 6
Step 6: Alice and Bob compute symmetric keys
        Alice:  ka = y^a mod p = 65536 mod 23 = 9
        Bob:    kb = x^b mod p = 216 mod 23 = 9
Step 7: 9 is the shared secret.
```